

Laporan Praktikum Kontrol Cerdas

Nama : Gunawan Wibisono
NIM : 224308083
Kelas : TKA – 7D
Akun *Github* (Tautan) : <https://github.com/gunawanwibisono83>
Student Lab Assistant : -

1. Judul Percobaan

Deep Reinforcement Learning untuk Kontrol Kompleks

2. Tujuan Percobaan

Tujuan dilakukannya percobaan ini yaitu :

- a. Mahasiswa dapat memahami konsep dasar *Deep Reinforcement Learning* (DRL) dalam kontrol sistem kompleks.
- b. Mahasiswa dapat mengimplementasikan agen DRL menggunakan algoritma *Deep Q-Network* (DQN) untuk kontrol otomatis.
- c. Mahasiswa dapat menganalisis performa DRL dibandingkan dengan metode kontrol konvensional.

3. Landasan Teori

Deep learning adalah salah satu bidang *machine learning* yang memanfaatkan banyak *layer* pengolahan informasi nonlinier untuk melakukan ekstraksi fitur, pengenalan pola, dan klasifikasi. Menurut Goodfellow, dkk. (2016), *deep learning* adalah sebuah pendekatan dalam penyelesaian masalah pada sistem pembelajaran komputer yang menggunakan konsep hierarki. Konsep hierarki membuat komputer mampu mempelajari konsep yang kompleks dengan menggabungkan dari konsep-konsep yang lebih sederhana (Rosita Dewi, 2018). Pada dasarnya *deep learning* adalah implementasi konsep dasar dari *machine learning* yang menerapkan algoritma ANN dengan lapisan yang lebih banyak (Alam dkk., 2019).

Reinforcement Learning (RL) adalah salah satu metode *machine learning* yang berfokus pada bagaimana agen harus mengambil tindakan dalam suatu lingkungan untuk memaksimalkan total hadiah yang diperolehnya. RL menggunakan mekanisme *trial and error*, di mana agen belajar dari konsekuensi tindakannya sendiri melalui umpan balik berbentuk *reward* atau *punishment* (Mehmood, 2024).

Deep Reinforcement learning (DRL) adalah perpaduan antara *Deep learning* dan *Reinforcement learning* yang menggabungkan keunggulan dari kedua paradigma tersebut untuk mempelajari tindakan yang optimal dalam lingkungan yang kompleks. Pada dasarnya, DRL menggunakan struktur jaringan saraf yang dalam untuk merepresentasikan dan memproses informasi dari lingkungan yang kompleks, dan menggunakan algoritma *reinforcement learning* untuk mengatur cara bagaimana agen memperbarui kebijakan atau nilai aksi berdasarkan pengalaman yang diperoleh. Dengan memanfaatkan kekuatan dari kedua paradigma ini, DRL dapat mempelajari tindakan yang optimal dalam berbagai tugas yang sulit dan kompleks (Nugraha, 2024).

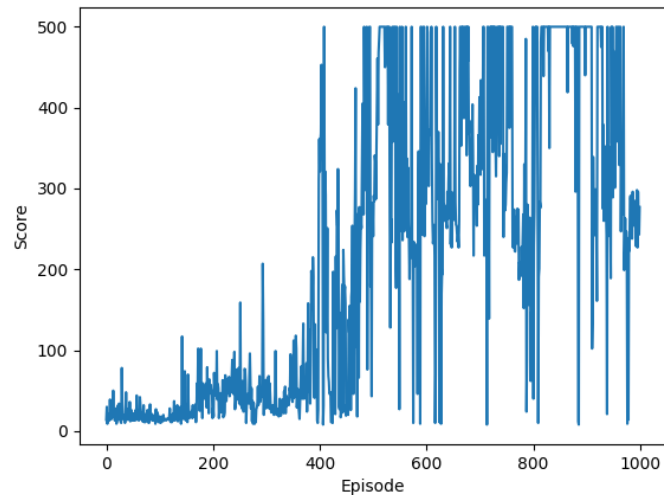
Deep Q-Network adalah sebuah algoritma dalam bidang *Deep Reinforcement learning* yang digunakan untuk mempelajari kebijakan (*policy*) optimal untuk pengambilan keputusan dalam lingkungan yang dinamis dan kompleks. DQN merupakan pengembangan dari algoritma *Q-learning* klasik dengan memanfaatkan jaringan saraf tiruan yang dalam (*Deep Neural networks*) untuk memodelkan fungsi nilai aksi (*action-value function*) yang kompleks. Pada dasarnya, DQN menggunakan jaringan saraf tiruan atau ANN untuk mengaproksimasi fungsi nilai aksi, yang memungkinkan agen untuk mengambil tindakan optimal berdasarkan keadaan saat ini dari lingkungan. DQN dilatih dengan menggunakan algoritma *Q-learning* dan menggunakan pengalaman yang diperoleh dari interaksi dengan lingkungan untuk memperbarui estimasi nilai aksi. Salah satu fitur utama dari DQN adalah penggunaan (*Replay Memory*) yang memungkinkan agen untuk menyimpan dan mengulangi pengalaman yang telah diperoleh selama proses pelatihan. Hal ini membantu dalam mengurangi korelasi antara pengalaman yang berurutan dan memungkinkan agen untuk lebih efisien mempelajari pola-pola dalam data (Nugraha, 2024).

4. Analisis

Dalam praktikum ini, praktika menerapkan salah satu bentuk pengembangan dari *Reinforcement Learning* (RL) yaitu *Deep Reinforcement Learning* (DRL) dengan pendekatan *Deep Q-Network* (DQN) untuk mengendalikan lingkungan simulasi *CartPole-v1* yang disediakan oleh *Gymnasium*. Lingkungan ini merepresentasikan sistem dinamis sebuah kereta yang membawa tiang di atasnya. Tujuan agen adalah menjaga keseimbangan tiang agar tidak jatuh dengan cara menggerakkan kereta ke kiri atau ke kanan. Agen mendapatkan *reward* positif setiap kali tiang tetap tegak dan episode akan berakhir ketika tiang jatuh atau kereta keluar dari batas lintasan.

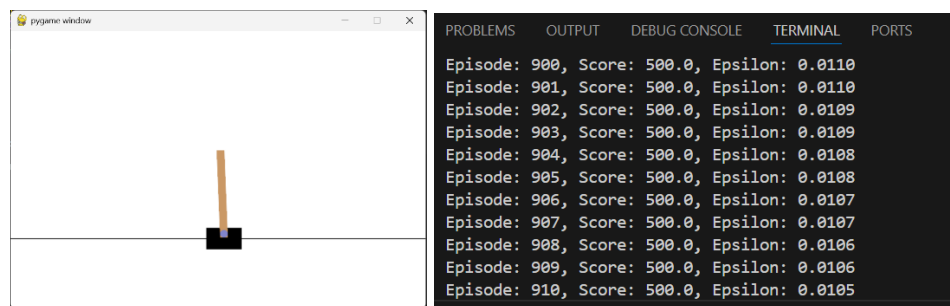
Proses kerja sistem ini terdiri dari beberapa tahapan utama. Pertama agen dibangun menggunakan arsitektur jaringan saraf tiruan (*neural network*) sederhana dengan *Keras Sequential Model* yang memprediksi nilai *Q-value* untuk setiap aksi berdasarkan keadaan (*state*) yang diterima. Kemudian agen menggunakan *replay memory* berbasis struktur data *deque* untuk menyimpan pengalaman interaksi sebelumnya dalam bentuk (*state, action, reward, next_state, done*). Pengambilan sampel secara acak dari memori ini (*experience replay*) membantu mencegah korelasi antar data dan meningkatkan stabilitas pembelajaran. Selanjutnya digunakan algoritma *epsilon-greedy* untuk menyeimbangkan antara eksplorasi (mencoba aksi baru secara acak) dan eksploitasi (memilih aksi terbaik berdasarkan pengalaman). Pada awal *training*, nilai epsilon masih tinggi agar agen banyak mengeksplorasi, kemudian secara bertahap menurun agar agen fokus pada aksi-aksi terbaik yang telah dipelajari.

Pada tahap *training*, sistem dilatih selama 1000 episode dengan ukuran *batch* 32. Parameter pembelajaran yang digunakan antara lain *learning rate* sebesar 0.001, *discount factor* (*gamma*) sebesar 0.95, dan nilai epsilon yang menurun secara bertahap dengan *epsilon decay* 0.995. Dari grafik hasil *training* terlihat dibawah, pada episode awal agen hanya mampu memperoleh skor rendah karena tindakan masih acak dan belum memiliki strategi yang baik. Namun setelah beberapa ratus episode, agen mulai menunjukkan pola gerakan yang lebih stabil dalam menjaga keseimbangan tiang. Nilai *reward* meningkat seiring berjalannya waktu hingga mendekati nilai maksimum.



Grafik Skor *Training* CartPole-v1

Selanjutnya, model diuji kembali pada lingkungan CartPole-v1 menggunakan mode visual (*render*) untuk melihat hasil performanya secara langsung. Pada tahap testing, nilai epsilon ditetapkan bernilai kecil agar agen lebih fokus pada strategi yang telah dipelajari daripada melakukan eksplorasi acak. Hasil pengujian menunjukkan bahwa agen mampu menjaga keseimbangan tiang selama beberapa ratus langkah pada sebagian besar episode, dengan rata-rata skor yang stabil di atas 400 poin dari maksimal 500 langkah. Hal ini membuktikan bahwa model DQN yang telah dilatih mampu mempelajari kebijakan optimal untuk menjaga keseimbangan sistem.



Tampilan *Testing* CartPole-v1

Secara keseluruhan, penerapan metode Deep Q-Network (DQN) dalam percobaan ini terbukti efektif untuk menyelesaikan permasalahan kontrol keseimbangan dinamis seperti CartPole. Kinerja agen dapat ditingkatkan dengan memperpanjang jumlah episode *training*, memperbesar kapasitas *replay memory*, atau menambahkan arsitektur jaringan yang lebih kompleks.

Tantangan utama selama pelatihan adalah waktu komputasi yang cukup lama karena model perlu melakukan ribuan iterasi pembaruan bobot agar mencapai stabilitas. Selain itu, keterbatasan perangkat keras (*hardware*) juga menjadi faktor pembatas dalam melakukan pelatihan dengan jumlah episode besar. Meskipun demikian, hasil akhir menunjukkan bahwa agen berhasil belajar secara mandiri bagaimana menyeimbangkan tiang dengan cukup efisien menggunakan konsep dasar *Deep Reinforcement Learning*.

5. Diskusi

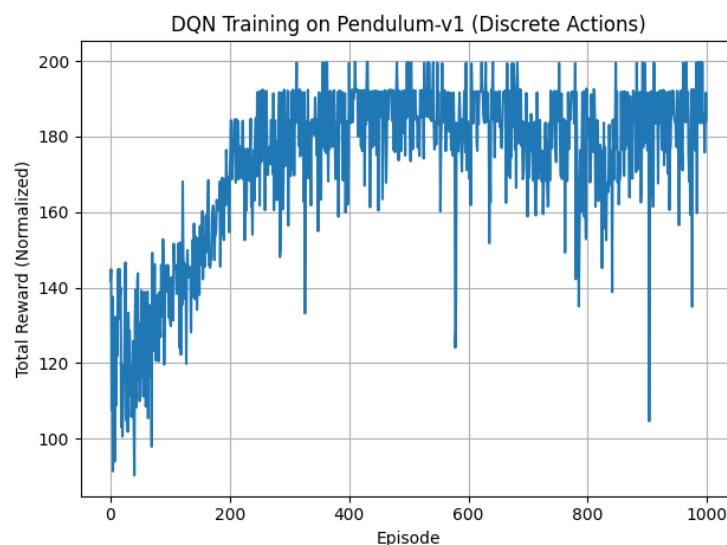
Walaupun praktika belum melakukan percobaan menggunakan *Q-Learning* klasik, namun berdasarkan beberapa literasi menunjukkan bahwa *Deep Q-Network* (DQN) memiliki beberapa kelebihan dibandingkan *Q-Learning* klasik. DQN mampu menangani masalah dengan jumlah state yang sangat besar atau bahkan *continue*, karena menggunakan jaringan saraf untuk memperkirakan nilai Q, sedangkan *Q-Learning* klasik hanya mengandalkan tabel Q-Value yang tidak efisien jika *state* terlalu banyak. Selain itu, DQN juga bisa melakukan generalisasi sehingga pengalaman pada kondisi tertentu dapat dipakai untuk kondisi lain yang mirip.

Untuk mengadaptasi metode DQN (*Deep Q-Network*) pada sistem kontrol dinamis seperti drone atau robot industri, terdapat beberapa hal penting yang perlu diperhatikan. *State* yang digunakan harus lebih kompleks karena sistem ini melibatkan banyak sensor seperti posisi, sudut, hingga data visual dari kamera. Kemudian karena aksi pada sistem nyata umumnya bersifat kontinu (misalnya kecepatan putar motor atau sudut kemudi) maka DQN perlu dikembangkan menjadi algoritma turunannya seperti DDPG atau algoritma yang lain yang mampu menangani aksi kontinu. Selain itu, fungsi *reward* harus dirancang dengan hati-hati agar benar-benar mencerminkan tujuan kontrol, misalnya menjaga kestabilan *drone* di udara atau mempertahankan posisi robot dengan presisi. Sistem ini juga memerlukan perangkat keras yang lebih kuat, karena proses komputasi DQN dan turunannya cukup intensif. Hal ini sejalan dengan hasil praktikum, di mana metode DRL terbukti memerlukan waktu pelatihan dan pengujian yang lebih lama dibandingkan metode RL klasik, meskipun dijalankan pada perangkat dengan spesifikasi yang sama.

6. Assignment

Setelah melakukan pelatihan agen pada *environment* CartPole-v1, praktikum dilanjutkan dengan penerapan algoritma DQN (*Deep Q-Network*) dan PID *Controller* pada *environment* lain yang lebih kompleks, yaitu Pendulum-v1. Berbeda dengan CartPole yang memiliki aksi diskret (ke kiri atau ke kanan), Pendulum-v1 merupakan sistem kontrol dengan aksi kontinu, di mana tujuan agen adalah menjaga batang pendulum agar tetap tegak di posisi atas (sudut nol radian). *Reward* pada *environment* ini bersifat negatif, dengan nilai maksimum 0 ketika pendulum benar-benar tegak. Hal ini membuat proses pembelajaran menjadi lebih sulit, karena agen harus menemukan aksi yang menghasilkan kehilangan energi minimal sambil menjaga posisi seimbang.

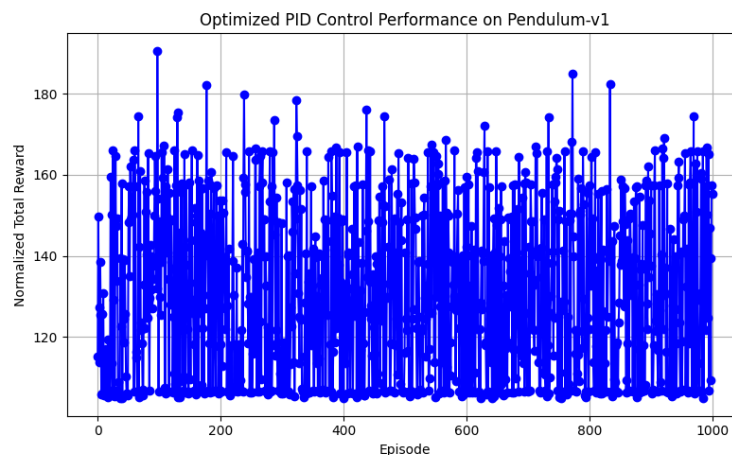
Dalam percobaan pertama, algoritma DQN digunakan untuk mengendalikan pendulum. Karena aksi kontinu tidak bisa langsung digunakan dalam DQN klasik, aksi tersebut didiskretisasi menjadi tiga nilai torsi: $[-2, 0, 2]$. Arsitektur jaringan saraf terdiri dari dua lapisan tersembunyi dengan 32 neuron beraktivasi ReLU, sedangkan fungsi *loss* menggunakan *Mean Squared Error* (MSE) dengan *optimizer* RMSProp. Agen dilatih selama 1000 episode dengan mekanisme *epsilon-greedy* untuk menyeimbangkan eksplorasi dan eksploitasi. Nilai epsilon menurun secara bertahap dari 1.0 ke 0.01 agar agen semakin fokus pada aksi terbaik seiring bertambahnya episode. *Reward* dinormalisasi agar proses pembelajaran lebih stabil.



Grafik Skor *Training* Pendulum-v1 algoritma DQN

Hasil *training* menunjukkan bahwa pada awal pelatihan agen belum mampu mengendalikan pendulum secara efektif, *reward* per episode masih rendah dan fluktuatif karena aksi yang masih acak. Namun setelah beberapa ratus episode, performa agen mulai meningkat dan *reward* total per episode menunjukkan nilai positif meskipun belum sepenuhnya stabil. Pola *reward* yang meningkat secara bertahap menunjukkan bahwa agen DQN mulai memahami hubungan antara torsi dan kestabilan sudut pendulum. Saat dilakukan *testing* dengan model terlatih selama 1000 episode, pendulum mampu bergerak menuju posisi tegak dan berosilasi di sekitar titik keseimbangan dengan cukup halus, walaupun sesekali kehilangan kestabilan akibat keterbatasan aksi diskret. Secara keseluruhan, DQN mampu menghasilkan kontrol yang adaptif, namun masih belum seefisien kontrol kontinu murni karena beberapa keterbatasan.

Selanjutnya dilakukan pengujian dengan metode PID (*Proportional–Integral–Derivative*) yang dikombinasikan dengan *energy-based control*. Strategi ini menggunakan dua mode kerja yaitu *energy control* untuk *fase swing-up* (mengayunkan pendulum dari bawah ke atas) dan PID *stabilizer* untuk menjaga keseimbangan di sekitar posisi tegak. Peralihan antara kedua mode dilakukan secara otomatis berdasarkan ambang sudut dan kecepatan sudut tertentu. Parameter yang digunakan adalah $K_p = 45.0$, $K_i = 0.02$, dan $K_d = 8.0$, dengan batas integral dan torsi agar sistem tetap stabil.

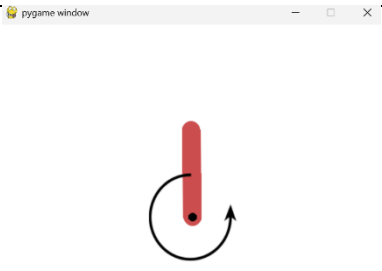
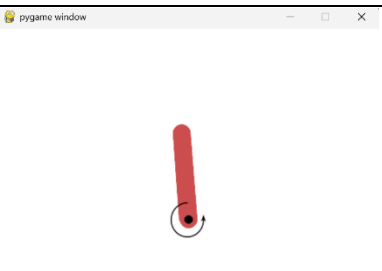


Grafik Skor *Training* Pendulum-v1 algoritma PID

Hasil simulasi menunjukkan bahwa kontroler PID mampu menempatkan posisi pendulum di atas sedikit lebih cepat setelah *fase swing-up* dan pergerakan

yang dihasilkan cenderung lebih halus dibandingkan DQN. Namun *reward* total yang diperoleh tidak stabil antar episode, menunjukkan bahwa praktika masih kurang optimal dalam mengimplementasikan algoritma PID dalam kontrol pendulum. Karena seharusnya kontroler PID memiliki keunggulan dalam kestabilan dan kecepatan respon karena berbasis model matematis walaupun memerlukan parameter yang tepat dan tidak dapat beradaptasi terhadap perubahan dinamika sistem tanpa penyesuaian manual.

7. Data dan Output Hasil Pengamatan

No.	Variabel	Hasil Pengamatan	
		Algoritma DQN	Kontrol PID
1.	Jumlah episode <i>training</i>	Proses <i>training</i> dilakukan sebanyak 1000 episode	Tidak melalui proses <i>training</i>
2.	Nilai Reward Episode Awal	<pre>Episode 1/20 selesai Total Reward: -130.01 Episode 2/20 selesai Total Reward: -128.20 Episode 3/20 selesai Total Reward: -127.60 Episode 4/20 selesai Total Reward: -130.36 Episode 5/20 selesai Total Reward: -130.40 Episode 6/20 selesai Total Reward: -385.09 Episode 7/20 selesai Total Reward: -128.67 Episode 8/20 selesai Total Reward: -361.57 Episode 9/20 selesai Total Reward: -251.71 Episode 10/20 selesai Total Reward: -130.74 </pre>	<pre>Memulai kontrol PID pada Pendulum-v1... Episode 1 selesai Total Reward: 157.68 Episode 2 selesai Total Reward: 105.19 Episode 3 selesai Total Reward: 156.42 Episode 4 selesai Total Reward: 158.98 Episode 5 selesai Total Reward: 164.86 Episode 6 selesai Total Reward: 140.78 Episode 7 selesai Total Reward: 130.26 Episode 8 selesai Total Reward: 170.87 Episode 9 selesai Total Reward: 109.48</pre>
3.	Nilai Reward Episode Akhir	<pre>Episode 882/1000 selesai Total Reward: -1.64 Episode 883/1000 selesai Total Reward: -1.64 Episode 884/1000 selesai Total Reward: -248.80 Episode 885/1000 selesai Total Reward: -2.04 Episode 886/1000 selesai Total Reward: -258.63 Episode 887/1000 selesai Total Reward: -127.84 Episode 888/1000 selesai Total Reward: -135.75 Episode 889/1000 selesai Total Reward: -129.17 Episode 890/1000 selesai Total Reward: -122.12 Episode 891/1000 selesai Total Reward: -130.39</pre>	<pre>Episode 996 selesai Total Reward: 146.83 Episode 997 selesai Total Reward: 139.38 Episode 998 selesai Total Reward: 109.24 Episode 999 selesai Total Reward: 157.48 Episode 1000 selesai Total Reward: 155.31 Semua simulasi selesai! Rata-rata reward akhir: 130.78 Reward terbaik: 190.67 Reward terendah: 104.75</pre>
4.	Kondisi Optimal		
5.	Kondisi Berhasil	Agen dapat mempertahankan pendulum dalam posisi tegak dengan skor mendekati 0.	Agen dapat mempertahankan pendulum dalam posisi tegak dengan skor mendekati 200.
6.	Kinerja Agen	Dengan proses <i>training</i> yang dilakukan sebanyak 1000 episode, agen dapat mempertahankan	Tanpa melalui proses <i>training</i> , agen mampu menempatkan pendulum dalam posisi tegak dan mencapai

		pendulum dalam posisi tegak dan mampu mencapai reward sebesar -3, -2, -1 atau mendekati 0.	reward sebesar 190 atau mendekati 200.
7.	Perbedaan Performa Agen	Awal pengujian agen belum mampu mengendalikan pendulum secara efektif, <i>reward</i> per episode masih rendah dan fluktuatif karena aksi yang masih acak. Namun setelah beberapa ratus episode, performa agen mulai meningkat namun <i>reward</i> total per episode belum sepenuhnya stabil.	Awal pengujian agen dapat mengendalikan pendulum dalam posisi tegak dengan waktu yang cepat, <i>reward</i> per episode cenderung tinggi namun fluktuatif. Namun kondisi tersebut tidak bertahan lama setelah beberapa episode, performa agen mulai menurun dengan <i>reward</i> total per episode yang masih sama.

8. Kesimpulan

Berdasarkan percobaan yang telah dilakukan, maka dapat diambil kesimpulan sebagai berikut:

- Mahasiswa telah berhasil mengimplementasikan agen DRL menggunakan algoritma *Deep Q-Network* (DQN) untuk mengendalikan lingkungan simulasi CartPole-v1 dan Pendulum-v1 yang disediakan oleh *Gymnasium*.
- Pada kontrol Pendulum-v1 dengan algoritma DQN, agen membutuhkan waktu yang cukup lama untuk belajar dan menunjukkan peningkatan performa secara bertahap. Nilai *reward* meningkat seiring bertambahnya jumlah episode, yang menunjukkan bahwa agen menemukan strategi kontrol yang optimal.
- Pada kontrol Pendulum-v1 menggunakan PID, sistem mampu dengan cepat memposisikan pendulum agar tegak di awal simulasi. Namun, seiring waktu nilai *reward* cenderung fluktuatif dan tidak meningkat secara signifikan. Hal ini menunjukkan bahwa pengendalian masih belum sepenuhnya stabil dan belum optimal untuk menjaga posisi tegak dalam jangka panjang.
- Secara keseluruhan, dapat disimpulkan bahwa DQN memiliki keunggulan dalam kemampuan belajar adaptif terhadap perubahan kondisi lingkungan, meskipun membutuhkan waktu pelatihan yang jauh lebih lama. Sebaliknya, PID lebih efisien dan cepat untuk sistem dengan parameter yang sudah diketahui, namun kurang fleksibel dalam menghadapi kondisi nonlinier atau lingkungan yang berubah-ubah.

9. Saran

Sebagai saran, pada percobaan selanjutnya disarankan untuk meningkatkan jumlah episode pelatihan agar algoritma DQN dapat lebih stabil dan menghasilkan kontrol yang optimal. Penyesuaian parameter seperti *learning rate*, *gamma*, serta arsitektur jaringan juga perlu dilakukan untuk meningkatkan performa pembelajaran. Pada kontrol PID, *tuning* parameter Kp, Ki, dan Kd secara sistematis penting dilakukan agar sistem lebih stabil dan responsif. Selain itu pengujian dapat dikembangkan pada sistem nyata atau dengan mengombinasikan metode PID dan DQN agar mampu beradaptasi terhadap perubahan kondisi lingkungan secara lebih efektif.

10. Daftar Pustaka

- Alam, I. F., Sarita, M. I., & Sajiah, A. M. (2019). IMPLEMENTASI DEEP LEARNING DENGAN METODE CONVOLUTIONAL NEURAL NETWORK UNTUK IDENTIFIKASI OBJEK SECARA REAL TIME BERBASIS ANDROID. *semanTIK*, 5(2), 237–244. <https://doi.org/10.5281/zenodo.3459374>
- Mehmood, S. (2024). *Reinforcement Learning for Automated Systems: Review of Concepts and Implementations*. 7.
- Nugraha, R. (2024). *PROGRAM STUDI TEKNIK INFORMATIKA FAKULTAS TEKNIK DAN ILMU KOMPUTER UNIVERSITAS KOMPUTER INDONESIA 2024*. Universitas Komputer Indonesia.
- Rosita Dewi, S. (2018). *DEEP LEARNING OBJECT DETECTION PADA VIDEO MENGGUNAKAN TENSORFLOW DAN CONVOLUTIONAL NEURAL NETWORK* [Universitas Islam Indonesia]. https://dspace.uii.ac.id/bitstream/handle/123456789/7762/14611242_Syarifah%20Rosita%20Dewi_Statistika.pdf?sequence=1&isAllowed=y